BIG DATA LAB INTERNAL - 2

Concise, Question-Aligned Spark and Hive Programs

Item	Included material
Questions	6 questions, exactly as supplied
Q1 API	Scala Spark DataFrame API
Q2-Q5 API	PySpark DataFrame API
Q6	Apache Hive installation and startup commands
Program style	Short code, simple comments, only required operations

Language note: Q1 explicitly says Scala API, so it is written in Scala. The supplied notebook examples for Q2-Q5 use PySpark, so those programs remain Python/PySpark. These are two different APIs; do not submit PySpark as the answer to Q1 if the examiner insists on Scala.

Input file: Keep textbook.txt in the same working folder as the Q2-Q4 programs, or change the path in `spark.read.text()`.

Hive note: The installation commands below are for Linux or WSL. Apache documentation says native Windows requires different installation steps.

1. QUESTION INDEX

Q.No.	Topic	Required operations
1	Spark Helpers - Scala API	Distinct users; average items per cart; grouping, aggregation and sorting.
2	Split textbook lines	Split lines into word arrays and use select().
3	Explode and text formatting	Explode arrays into rows; apply text formatting functions.
4	Word frequency	groupBy() and count() to calculate word frequency.
5	CSV storage	Save the analyzed word-frequency result in CSV format.
6	Install and run Hive	Check prerequisites, configure Hive, initialize metastore and connect.

Common Spark sequence

Create or obtain a SparkSession -> read the text file -> apply DataFrame transformations -> call show() or write.csv() to produce the result.

Remember

Spark transformations such as select(), explode() and groupBy() build a new DataFrame. Actions such as show() and write.csv() display or save the result.

2. QUESTION 1 - SPARK ACTIVITY USING SCALA API

Objective

Create sample user activity data and find distinct users, average items per cart, and grouped/aggregated/sorted results.

Program (Scala)

```
import org.apache.spark.sql.SparkSession
import org.apache.spark.sql.functions._

case class Activity(userId: String, cartId: String, itemId: String)

object ActivityAnalysis {
  def main(args: Array[String]): Unit = {
    val spark = SparkSession.builder()
      .appName("Activity Analysis").master("local[*]").getOrCreate()
    import spark.implicits._

    // Sample user activity
    val df = Seq(
      Activity("U1", "C1", "I1"), Activity("U1", "C1", "I2"),
      Activity("U1", "C2", "I3"), Activity("U2", "C3", "I4"),
      Activity("U2", "C3", "I5")
    ).toDF()

    // a) Distinct users
    df.select("userId").distinct().show()

    // b) Average items per cart
    val carts = df.groupBy("cartId").agg(count("itemId").alias("items"))
    carts.agg(avg("items").alias("averageItems")).show()

    // c) Group, aggregate and sort by total items
    df.groupBy("userId")
      .agg(countDistinct("cartId").alias("carts"),
           count("itemId").alias("totalItems"))
      .orderBy(desc("totalItems")).show()

    spark.stop()
  }
}
```

Key functions: `distinct()` gives unique users; `groupBy()` creates cart/user groups; `count()` counts items; `avg()` calculates the average; `orderBy(desc(...))` sorts from highest to lowest.

Run: Save as `ActivityAnalysis.scala` and submit/run it in the Scala Spark environment configured by your lab.

3. QUESTION 2 - SPLIT TEXTBOOK LINES USING select()

Objective

Read each textbook line, split it into an array of words, and use select() to obtain the word array and first word.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, split, trim

spark = SparkSession.builder.appName("SplitText").getOrCreate()
text_df = spark.read.text("textbook.txt")

# Split each line into an array of words
words_df = text_df.select(
    split(trim(col("value")), r"\s+").alias("words"))
words_df.show(truncate=False)

# Select the first word from each line
first_df = text_df.select(
    split(trim(col("value")), r"\s+").getItem(0).alias("first_word"))
first_df.show()
spark.stop()
```

4. QUESTION 3 - EXPLODE ARRAYS AND FORMAT TEXT

Objective

Convert each word array into separate rows and apply simple text formatting.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, split, trim, explode
from pyspark.sql.functions import upper, lower, initcap, length

spark = SparkSession.builder.appName("ExplodeText").getOrCreate()
text_df = spark.read.text("textbook.txt")

# Convert arrays into individual word rows
words_df = text_df.select(
    explode(split(trim(col("value")), r"\s+")).alias("word"))

# Apply text formatting functions
result = words_df.select(
    "word", upper("word").alias("upper"),
    lower("word").alias("lower"),
    initcap("word").alias("capitalized"),
    length(trim("word")).alias("length"))
result.show(truncate=False)
spark.stop()
```

Key functions: `split()` makes an array; `explode()` makes one row per word; `upper()`, `lower()`, `initcap()` and `length()` format or describe each word.

5. QUESTION 4 - WORD FREQUENCY

Objective

Find how many times each word occurs in the textbook using groupBy() and count().

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, lower, regexp_replace, split, explode, desc

spark = SparkSession.builder.appName("WordFrequency").getOrCreate()
text_df = spark.read.text("textbook.txt")

# Make words lowercase and remove punctuation
clean = regexp_replace(lower(col("value")), r"[^a-z0-9\s]", "")
words = text_df.select(explode(split(clean, r"\s+")).alias("word"))
words = words.filter(col("word") != "")

# Count and sort words by frequency
freq_df = words.groupBy("word").count().orderBy(desc("count"))
freq_df.show(truncate=False)
# Keep this Spark session open for Question 5.
```

6. QUESTION 5 - STORE RESULTS IN CSV

Objective

Save the word-frequency DataFrame created in Question 4.

```
# Run after Question 4, while freq_df and spark still exist.
# Spark writes a folder containing one CSV part file.
freq_df.coalesce(1).write.mode("overwrite") \
    .option("header", "true").csv("word_frequency_csv")
print("CSV results saved in word_frequency_csv/")
spark.stop()
```

Important: Run Q5 after Q4 in the same Spark session. Spark writes a folder named `word_frequency_csv` containing a `part-*.csv` file and a success marker. `coalesce(1)` requests one output part; it does not create a single CSV path by itself.

7. QUESTION 6 - INSTALL AND RUN APACHE HIVE

Objective

Verify Java and Hadoop, configure Hive, initialize the metastore and start Hive for queries.

> Prerequisites: Java and Hadoop must already be installed and working. Download the binary Hive archive from the official Apache Hive release page. The commands below use a Linux/WSL shell.

Installation and startup

```
# 1. Check Java and Hadoop
java -version
hadoop version

# 2. Download the binary Hive archive from the Apache Hive website,
#    then extract it (replace x.y.z with the downloaded version).
tar -xzf apache-hive-x.y.z-bin.tar.gz

# 3. Set Hive environment variables (Linux / WSL terminal)
export HIVE_HOME="$HOME/apache-hive-x.y.z-bin"
export PATH="$HIVE_HOME/bin:$PATH"
cd "$HIVE_HOME"

# 4. Create the local Derby metastore schema (first-time setup)
schematool -dbType derby -initSchema

# 5. Start HiveServer2 (keep this terminal open)
hiveserver2
```

Connect and test

```
# Open a SECOND terminal; set the Hive path in this terminal too
export HIVE_HOME="$HOME/apache-hive-x.y.z-bin"
export PATH="$HIVE_HOME/bin:$PATH"
beeline -u jdbc:hive2://localhost:10000

# At the beeline> prompt, test Hive
SHOW DATABASES;

# Exit Beeline with !quit; use legacy CLI only if your lab requires it
hive
```

> Remember: Run `hiveserver2` in the first terminal and keep it running. Run `beeline` in a second terminal. Apache marks the old Hive CLI as deprecated in favor of Beeline. If your lab specifically expects the legacy CLI, use `hive` as shown in the last line.

Windows users: These are Linux/WSL commands, not PowerShell commands. Apache notes that native Windows setup uses different steps.

8. FINAL QUICK REFERENCE

Question	Remember this sequence
1 - Scala Activity	SparkSession -> sample Activity data -> distinct users -> count items/cart -> average -> group/aggregate/sort
2 - Split text	read.text() -> split() -> select() -> show()
3 - Explode/format	split() -> explode() -> upper/lower/initcap/length -> show()
4 - Word frequency	read.text() -> split words -> explode() -> groupBy("word").count() -> sort
5 - CSV	freq_df.coalesce(1).write.option("header", true).csv(path)
6 - Hive	check Java/Hadoop -> extract Hive -> HIVE_HOME/PATH -> schematool -> hiveserver2 -> beeline

Common file and runtime checks

1. Put textbook.txt in the program folder or use its full path. 2. Start Spark with a working Spark installation. 3. Run Q4 before Q5 because Q5 saves freq_df from Q4. 4. Start HiveServer2 before connecting with Beeline.

Official technical references

Apache Spark Scala Dataset API: https://spark.apache.org/docs/latest/api/scala/org/apache/spark/sql/Dataset.html

Apache Spark PySpark split API: https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.functions.split.html

Apache Hive Getting Started: https://hive.apache.org/development/gettingstarted-latest/

Source alignment: program topics and operations are based on the six questions and the supplied notebook images. No extra Hive database/table/view/function exercises have been added.